ReproAgent: Project Report (Mid-term)

Nov 6, 2026 · @Vaishali

Summary

ReproAgent is a working prototype. It turns a written bug report into a failing test, then proposes a patch and proves the patch makes that test pass, all with a small open-weight model on an 8 GB laptop.

Team: Keshav Dubey (2025204041), Rishabh Singh (2025204025), Aasritha Kalluri (2025204019). **Course:** AutoSE, Prof. Karthik Vaidhyanathan.

What works today:

- **Reproduction.** On 20 benchmark bugs, the full agent wrote a *valid* reproduction test for 14 (70%). The simple baseline managed 6 (30%).
- **Fixing** (added on the instructor's request). Of the 17 bugs it reproduced, it fixed 7 correctly (41%). Every fix was confirmed by a hidden check the agent never sees.
- **Multi-location fixing.** A new mode keeps partial fixes and follows the bug to its next location. It fixed 2 of 4 reproduced two-place bugs. The previous fixer fixed none.
- **Interfaces.** A command-line tool and a local web UI that shows every saved result, runs the agent live, and summarises the evaluation.

The model is `qwen2.5-coder:3b`, running in Ollama on the CPU. One bug takes about 1–3 minutes.

The problem and our goal

Before a developer can fix a bug, they must first make it happen on their own machine. That step is slow and often skipped. Bug reports describe symptoms in words ("the discount is applied to small orders"), not as code someone can run.

Our goal is an assistant that does that first step automatically, and then the fix. The constraints come from the course:

- **Small, open model.** It must run locally with open weights, no cloud API. We use a 3-billion-parameter coding model.
- **Ordinary laptop.** 8 GB of RAM and no usable GPU, so everything runs on the CPU.
- **Honest evaluation.** Real bugs from BugsInPy plus seeded bugs. The agent is never shown the answer.

The project grew in three steps:

1. **Proposal:** reproduce bugs only; fixing was out of scope.
2. **Instructor's request:** add bug resolution. Fixing became a second phase that starts only after a reproduction is accepted, because the reproduction test tells the fixer where to look and whether a fix works.
3. **Multi-location fixing and a web UI:** real fixes often touch more than one place, so we added a mode that builds a fix step by step. The web UI makes the work easy to demonstrate.

Architecture

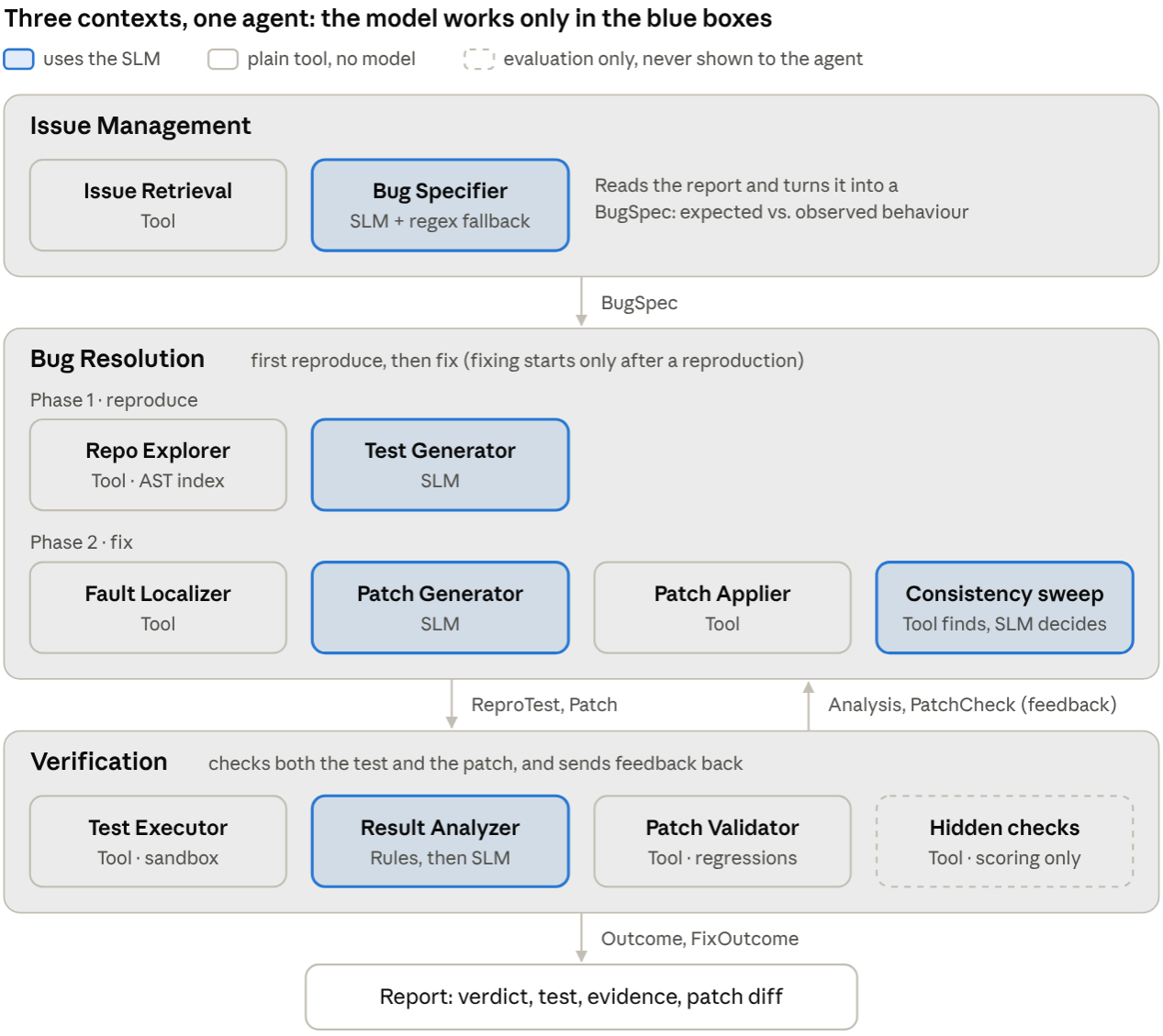
We derived the architecture with the course's method: use case, then bounded contexts, shared models, hidden modules, their classification, and finally how many agents. The result: **three contexts, twelve modules, one agent.** The small model is used only in five narrow places; everything else is plain, testable code.

1. Use case

The course's reference use case is *autonomous bug fixing*. ReproAgent covers every step, and adds an explicit, checkable reproduction step before the fix.

Bug-fixing step (slides)	What ReproAgent does
Understand the bug	Reads the report; extracts expected vs. observed behaviour
Explore the repository	Searches the project's code index for relevant functions
New: make the bug executable	Writes a pytest test, runs it in a sandbox, refines it from the result
Identify the root cause	Locates the fault from the failing test: traceback, called functions, constants they read
Implement a fix	The model rewrites the suspicious definition(s)
Validate the fix	The test must now pass, and existing tests must not break
Report the outcome	Verdict, test, evidence, patch diff

2–5. Bounded contexts, shared models, modules and their classification



ReproAgent architecture · 3 bounded contexts, 12 modules, shared models on the arrows

The picture reads top to bottom. Each context hands the next one a shared model (the words on the arrows). Verification sends feedback back up, which drives the retry loops.

Shared model	From → to	What it carries
BugSpec	Issue Management → Bug Resolution	Expected and observed behaviour, symbols, example inputs, "is this actionable?"
ReproTest	Bug Resolution → Verification	The generated test code
Patch	Bug Resolution → Verification	Changed definitions, a unified diff, patched files

Shared model	From → to	What it carries
Analysis, PatchCheck	Verification → Bug Resolution	Category (e.g. invalid test, still failing, partial progress) and the feedback text
Outcome, FixOutcome	Verification → user	Verdict, final test, evidence, fix status, diff, time, number of model calls

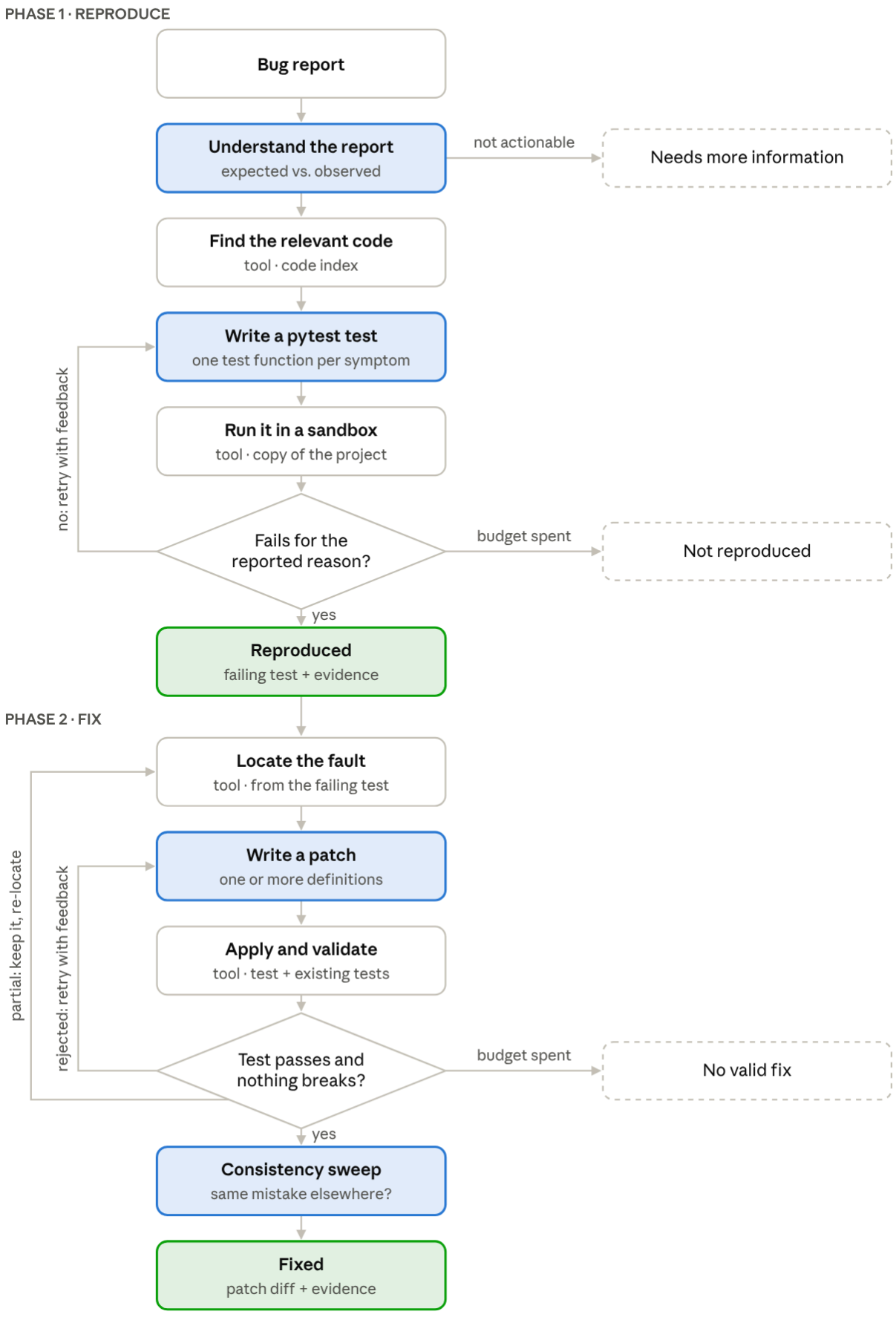
6. How many agents?

Option	For	Against, on our setup
1 agent, fixed workflow (chosen)	One model in memory; small focused prompts; reproducible behaviour; every ablation is a switch; the fixer reuses the reproducer's state	Less flexible than free-form tool calling
2 agents (reproducer + fixer)	Clean split of the phases	Both would be the same 3B model on 8 GB; hand-offs must copy context; fixing waits for reproduction anyway
3 agents (one per context)	Matches the contexts	Hand-off overhead; 3B models are unreliable at free-form multi-agent tool use

We chose **one agent with a scripted two-phase workflow**. The tools are ordinary Python functions that the workflow calls, so the model never has to choose a tool, a task 3B models do badly. Only one model fits in 8 GB, and fixing depends strictly on the reproduction, so a second agent would add no parallelism.

How a bug is handled, end to end

Every bug goes through the same fixed workflow. Fixing starts only after the agent has a test that fails for the reported reason.



ReproAgent workflow · phase 1 reproduce, phase 2 fix, with feedback loops

Phase 1, reproduce. The model reads the report and writes down what *should* happen and what *does* happen. A code index finds the functions involved. The model then writes a pytest test, which runs in a throw-away copy of the project. If the test cannot run, passes, or fails for an unrelated reason, the agent gets specific feedback and tries again, up to 4 times. Example feedback: "`cart.products` does not exist; the class has `items`."

Phase 2, fix. The failing test points to the suspicious code: functions in the error traceback, the functions whose result the test checks, and the constants they read. The model rewrites those definitions. The patch is applied in a fresh copy. It is accepted only if the reproduction test now passes and the project's related existing tests still pass. Otherwise the agent explains what went wrong and tries again, up to 3 times. The user's project is never modified.

Multi-location fixes (`fix_multi` mode). Some bugs need changes in two places. The plain fixer throws away a half-fix and starts over. `fix_multi` instead:

1. Writes one test function per symptom in the report.
2. Lets a patch change several definitions or files at once.
3. Keeps a patch that fixes some test functions without breaking anything. It then locates the fault again on the patched code, using only the still-failing tests, and hides the already-fixed code so it cannot be undone. Progress earns one extra attempt (at most 5).
4. After success, runs a consistency sweep: it looks for the same mistake in similar code the test does not cover.

Worked example: the course's discount bug. The report says a ₹800 order wrongly gets a 10% discount. The agent's test adds one ₹800 product, checks out, and asserts the discount is 0. On the buggy code it fails with `Decimal('80.00') == 0`, exactly the reported symptom. The fault locator follows `checkout` → `bulk_discount` → `DISCOUNT_THRESHOLD` (set to 100 instead of 1,000). The second patch makes `bulk_discount` compare against ₹1,000: the test passes, existing tests pass, and the hidden check confirms it is correct.

Benchmarks and how we score

We test on 26 bugs in three sets. Every bug has a buggy and a fixed version. The fixed version is used only for scoring, never shown to the agent.

Set	Bugs	Where they come from	Why it is there
BugsInPy	12	Real bugs from youtube-dl (7), thefuck (4) and tqdm (1)	Real-world bugs, with bug reports rewritten to describe only the symptom
ShopCart	8 + 1 vague	A small online-shop app we wrote, with seeded bugs	Includes the course's "discount below ₹1,000" example; the vague report should get "needs more information"
Multi-location	5	Seeded in ShopCart	Each needs changes in two places; a script checks that fixing only one place is not enough

How we decide whether the agent succeeded:

- **Valid reproduction.** The test fails on the buggy version *and* passes on the fixed version. This catches tests that fail for the wrong reason.
- **Plausible fix.** The agent's own checks accept the patch: the reproduction test now passes and the project's existing tests still pass.
- **Correct fix.** Plausible *and* it passes a hidden check written for each bug. The hidden check tests the intended behaviour more broadly, for example boundary values, so a patch that only special-cases the test's input is caught.

A second score: valid by meaning. Strict fail-to-pass also rejects a test that only words an expected error message differently from the real code ("quantity must be positive" vs. "quantity cannot be negative"). We keep the strict score as the headline and add a second one. When the *only* failures on the fixed version are message-wording differences, three layers decide whether the messages mean the same:

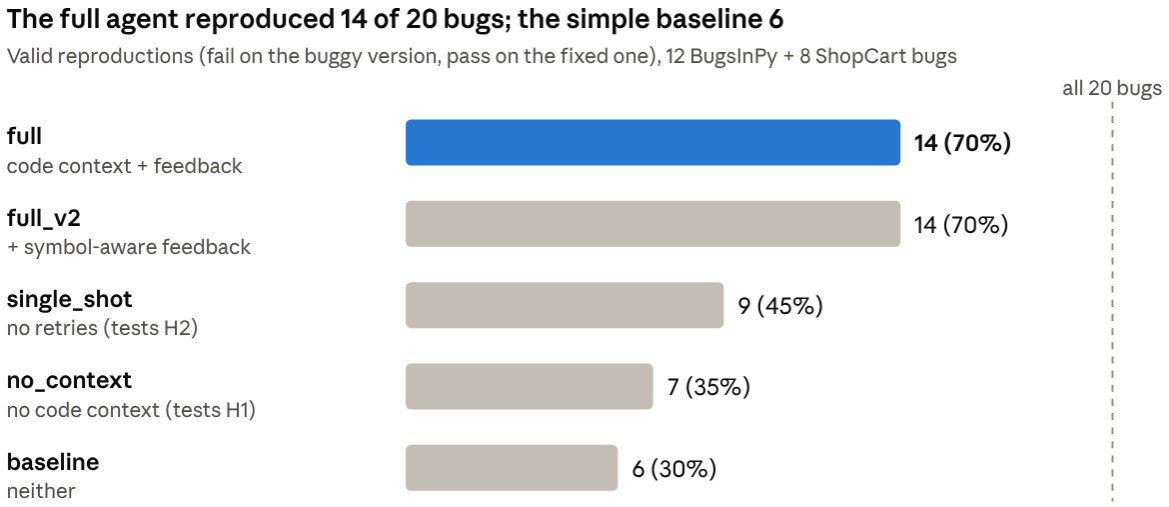
1. **Report rule:** if the bug report is about the message itself, the wording must match exactly.
2. **Text rule:** one message contains the other.
3. **Model judge:** the local model is asked whether both messages state the same rule or problem.

The judge's prompt was chosen on a separate set of 14 message pairs, none from the benchmark. It got 14/14 right, including opposite-meaning traps such as "must be positive" vs. "must be negative".

Results

The full agent more than doubles the baseline at reproduction, and the reproduction test is what makes its fixes trustworthy. All numbers are from one run per setting.

Reproduction

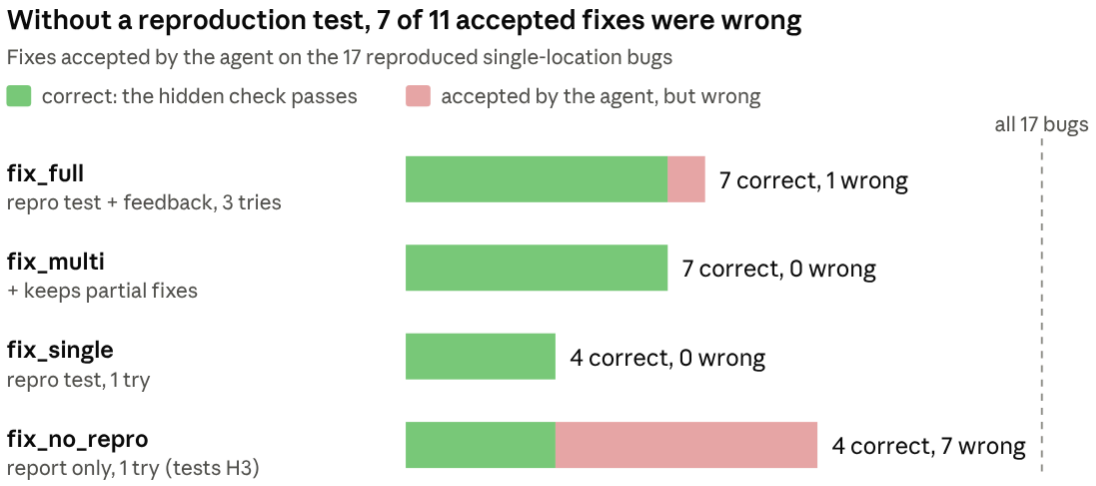


results/SUMMARY.md · one run per mode, qwen2.5-coder:3b on CPU

When the full agent said "reproduced", it was right 14 times out of 15 (93%). The one false positive is explained under failures. A valid reproduction takes about 60–90 seconds. The vague report ("the website doesn't work properly") correctly got *needs more information*.

Of all tests that failed the strict check, exactly one did so only because of message wording: `ml_quantity`. The judge accepted it, so multi-location reproduction is 1 of 5 strict and 2 of 5 by meaning. All other false positives are real test mistakes, and the headline numbers above are unchanged.

Fixing



results/FIX_SUMMARY.md · one run per mode, hidden checks in benchmarks/*/oracle_checks.py

With the reproduction test, the agent's own "fixed" verdict can be trusted: 7 of 8 accepted patches were correct in `fix_full`, and 7 of 7 in `fix_multi`. Without the test, the agent accepted 11 patches and only 4 were correct. A fix takes about 2.5 minutes in `fix_full`.

Multi-location bugs

On the 4 multi-location bugs that the agent reproduced, `fix_multi` fixed 2 correctly; `fix_full` fixed none. Both successes followed the intended path:

- `ml_boundaries`: patch 1 fixed `bulk_discount` and was kept; re-localisation pointed to `shipping_fee`; patch 2 completed the fix.
- `ml_quantity`: patch 1 fixed `Cart.set_quantity`; patch 2 fixed `Cart.add`.

The other two failed because their reproduction tests were wrong, so no correct fix could pass them. Reproduction is now the bottleneck for multi-symptom reports: only 1 of the 5 got a valid test.

What this says about our hypotheses

Hypothesis	Comparison	Result	Verdict
H1: showing the model the project's code helps	`full` vs. `no_context`	14 vs. 7 valid reproductions	Supported

Hypothesis	Comparison	Result	Verdict
H2: execution feedback and retries help	`full` vs. `single_shot`; `fix_full` vs. `fix_single`	14 vs. 9 reproductions; 7 vs. 4 correct fixes	Supported
H3: reproducing first makes fixes more often correct	`fix_full` vs. `fix_no_repro`	7 of 8 accepted fixes correct vs. 4 of 11	Supported, mainly by making the "fixed" verdict trustworthy
Multi-location repair helps two-place bugs	`fix_multi` vs. `fix_full`	2 of 4 vs. 0 of 4; no loss on single-location bugs (7 vs. 7)	Supported, on a small sample

With 20 bugs and one run, differences of 1–2 bugs are within run-to-run noise. Repeating every run with 3 seeds is part of the remaining work.

Web UI and the demo

The web UI runs locally with no extra installs: `python -m reproagent.webui`, then open http://127.0.0.1:8000. It has three tabs.

Tab	What you can do	Needs the model?
Our tested results	Pick any of the 26 evaluated bugs plus a reproduction mode and a fix mode. See the bug report, how the agent understood it, the test, the pytest evidence, the patch as a coloured diff, the changed locations, and the hidden-check verdict	No, opens instantly
Run on a new bug	Paste a bug report or a GitHub issue URL, give the project folder, tick "Also fix the bug", and watch the agent's log live. Results are saved to `results/webui/`	Yes (1–5 minutes on CPU)
Evaluation summary	All result tables, plus a per-bug grid. Clicking any cell opens that exact result	No

Suggested demo order: the summary tab for the numbers; then `ml_boundaries` with `fix_multi`, to show a two-place fix built in two steps; then a live run of the discount bug.

The command-line tool does the same as the live tab: `python -m reproagent --repo <project> --issue <report> --fix`. Neither interface modifies the user's project; the patch is reported as a diff.

What went wrong: failures and limitations

The biggest single problem is the small model inventing names. Most failed attempts are tests or patches that cannot run, not wrong ideas.

Where	Failure	Example	What we did / plan
Reproduction	**Invented imports and attributes** (26 of 45 attempts in `full` mode could not run)	`cart.products` instead of `cart.items`; importing a module that does not exist	`full_v2` answers these errors with the real names from the code index; it fixed two thefuck bugs
Reproduction	**Test fails for the wrong reason** (false positive)	`shop_shipping`: arguments in the wrong order, but the wrong total happened to match the report	Caught by the fail-to-pass check; planned: constructor signature hints
Reproduction	**Test asserts the bug instead of the fix**	`shop_remove_missing`, `ml_coupon`	Planned: a static check before running
Reproduction	**The 3B model as a judge was unreliable**	It accepted a wrong subtotal and rejected a correct test	Replaced by deterministic evidence matching; the model is now only a fallback
Fixing	**Patch misses the root cause** (21 of 40 attempts)	`youtube-dl_3`, `_27`, `_29`, `shop_gst`	Planned: show the failing line and values; try a 7B model
Fixing	**Fix accepted by our checks but wrong**	Live discount run turned `>=` into `<`	Only the hidden check caught it; planned: run the project's full test suite

Where	Failure	Example	What we did / plan
Fixing	**Bad reproduction leads to a bad fix**	`shop_shipping` : the "fix" set free shipping to Rs 0	Better reproduction precision also improves fixing
Multi-location	**Multi-symptom reports are hard to reproduce**	Only 1 of 5 got a valid test	Planned: check each test function separately and keep the valid ones
Our own code	**Harness bugs** found by the evaluation	New imports inserted inside a multi-line import; a half-fix on the last attempt left no budget; the model undoing a kept fix	All fixed, unit-tested and the affected runs re-run

Limitations to keep in mind:

- Small sample (26 bugs) and one run per setting. A difference of 1–2 bugs is within run-to-run noise.
- One model, CPU only.
- The bug reports are our own symptom-only rewrites.
- youtube-dl may be in the model's training data.
- The sandbox is a temporary copy plus a subprocess, not a container.

Changes from the proposal

Change	Proposal said	What we did, and why
Bug fixing	Out of scope	Added on the instructor's request, as a second phase that depends on the reproduction. Adds hypothesis H3: a reproduction test makes a small model's fixes more often correct
Multi-location fixing	Not planned	Added because real fixes often touch several places, and our first fixer only succeeded when the model got every place right in one answer

Change	Proposal said	What we did, and why
Interface	A simple interface	A command-line tool and a local web UI with saved results, live runs and an evaluation summary
Judging a failing test	An LLM decides whether a failure matches the report	The 3B judge was unreliable both ways, so a deterministic evidence check runs first; the model is only a fallback
Benchmark	20–30 bugs	25 bugs (12 real + 13 seeded) + 1 vague report. Heavy projects such as pandas cannot run at old commits on 8 GB
Environment	—	The laptop has Python 3.14 while BugsInPy targets 3.6–3.8. Two tiny compatibility shims were added; one bug was excluded (`thefuck_18`). All documented
Hardware	GPU if possible	The GPU driver is too old for current Ollama, so all numbers are CPU-only (about 20–40 s per model call)

Remaining work to the final submission

Four items remain for the final submission.

- [ ] **Model comparison:** qwen2.5-coder 1.5B / 3B / 7B and qwen3:8b; needs a GPU driver update.
- [ ] **Optional Docker sandbox.**
- [ ] **Grow the benchmark to about 30 bugs**, including real multi-location bugs from BugsInPy, and **repeat every run with 3 seeds**.
- [ ] **Reduce the failures and limitations** described above, so the agent makes fewer mistakes and the evaluation is stronger.